

KONYA FOOD AND AGRICULTURE UNIVERSITY
FACULTY OF ENGINEERING AND ARCHITECTURE
DEPARTMENT OF COMPUTER ENGINEERING

**LIGHTWEIGHT HYBRID INTRUSION DETECTION
SYSTEM FOR SMALL-SCALE NETWORKS**

SYSTEM DESIGN DESCRIPTIONS
(SDD)

Mustafa Emir KAYNAR
&
Şamil KEKLİKOĞLU

Date of Issue: January 14, 2026

Status: Version 1.0

Issuing Organization: Konya Food and Agriculture University, Department of Computer Engineering

Authorship: Mustafa Emir KAYNAR & Şamil KEKLİKOĞLU

Change History:

Version	Date	Authorship	Change Description

Table of Contents

Table of Contents.....	3
1. Introduction.....	5
1.1 Purpose.....	5
1.2 Scope.....	5
1.3 Definitions and Acronyms.....	5
1.4 References.....	6
2. System Overview.....	6
3. System Architecture.....	7
3.1 Architectural Design.....	7
3.2 Decomposition Description.....	7
3.2.1 Packet Capture & Preprocessing.....	7
3.2.2 Flow Feature Extraction Engine.....	7
3.2.3 Rule-Based Detection Engine.....	8
3.2.4 Storage & Visualization.....	8
3.3 Design Rationale.....	8
3.3.1 Hybrid Detection Strategy (Phased Approach).....	8
3.3.2 Why Flow-Based Analysis (ETA)?.....	8
3.3.3 Why Python & Scapy?.....	8
3.3.4 Why InfluxDB & Grafana?.....	9
4. Data Design.....	9
4.1 Data Description.....	9
4.2 Data Dictionary.....	9
4.3 Database Schema Strategy (InfluxDB).....	10
4.4 Mathematical Model of Feature Extraction.....	10
4.4.1 Inter-Arrival Time (IAT).....	10
4.4.2 Flow Throughput (Rates).....	11
4.4.3 TCP Flag Ratios.....	11
5. Component Design.....	11
5.1 System Class Diagram.....	12
5.2 Component Descriptions.....	12
5.2.1 Class: PacketSniffer.....	12
5.2.2 Class: FlowProcessor.....	12
5.2.3 Class: RuleEngine.....	13
5.2.4 Class: DatabaseManager.....	13
5.2.5 Class: MLEngine (Future/Conditional).....	13
5.3 Dynamic Behavior (Sequence Diagram).....	13
5.4 Algorithmic Specifications.....	14
5.4.1 Algorithm 1 Main Packet Processing Loop.....	14
5.4.2 Algorithm 2 Rule-Based Inspection Logic.....	15
6. Human Interface Design.....	16
6.1 Overview of User Interface.....	16

6.2 Screen Images.....	16
6.3 Screen Objects and Actions.....	16
6.3.1 Panel A: Network Throughput & Latency.....	16
6.3.2 Panel B: Recent Alerts Log.....	16
6.3.3 Panel C: Protocol Distribution.....	17
7. Requirements Matrix.....	17
8. Appendices.....	18
9. Test Strategy and Validation.....	18
9.1 Test Environment Setup.....	18
9.2 Attack Simulation Tools & Scenarios.....	19
9.2.1 Scenario A: Port Scanning Detection.....	19
9.2.2 Scenario B: DoS / SYN Flood Detection.....	19
9.2.3 Scenario C: Network Stress Testing (Benign Traffic).....	19
9.3 Performance Evaluation Metrics.....	20
9.4 Validation Metrics.....	20

1. Introduction

1.1 Purpose

The purpose of this System Design Description (SDD) document is to detail the architectural design, components, and data structures of the "Lightweight Rule-Based Intrusion Detection System for Small-scale Networks". This document provides a technical guide for the development of a resource-efficient security tool. It is intended for the development team and the project advisor to ensure a shared understanding of the system's rule-based detection mechanisms and operational logic.

1.2 Scope

The system is designed as a real-time network monitoring and intrusion detection tool optimized for small-scale networks. The scope includes:

- **Traffic Capture:** Real-time capture of network packets from a single interface using Python libraries (Scapy/pcapy).
- **Statistical Flow Analysis:** Calculating flow metrics (packet counts, flags, duration) to detect volumetric attacks.
- **Rule-Based Detection:** A deterministic engine that flags suspicious activities based on predefined thresholds and signatures (e.g., Port Scanning, SYN Flood).
- **Visualization:** Reporting of detected threats via an interactive web-based dashboard (Grafana).

Out of Scope:

- Deep Packet Inspection (DPI) for payload analysis.
- Automated intrusion prevention (blocking traffic).
- Decryption of SSL/TLS encrypted traffic.

1.3 Definitions and Acronyms

IDS (Intrusion Detection System): Software that monitors a network for malicious activity.

Rule-Based Detection: A detection method that compares traffic patterns against a set of predefined rules or thresholds.

Flow Features: Statistical properties extracted from a stream of packets (e.g., packet count per second, flag distribution).

ETA (Encrypted Traffic Analysis): Analyzing encrypted traffic using metadata (size, timing) without decryption.

1.4 References

- El-Taj, H., et al. (2025). A Lightweight Network Intrusion Detection System for SMEs.
- Almalawi, A. (2025). A Lightweight Intrusion Detection System for Internet of Things: Clustering and Monte Carlo Cross-Entropy Approach. *Sensors* 2025, 25, 2235.
- CIC-IDS-2017/18 Dataset Documentation.
- Scapy Documentation (<https://scapy.net/>).
- InfluxDB & Grafana Official Documentation.

2. System Overview

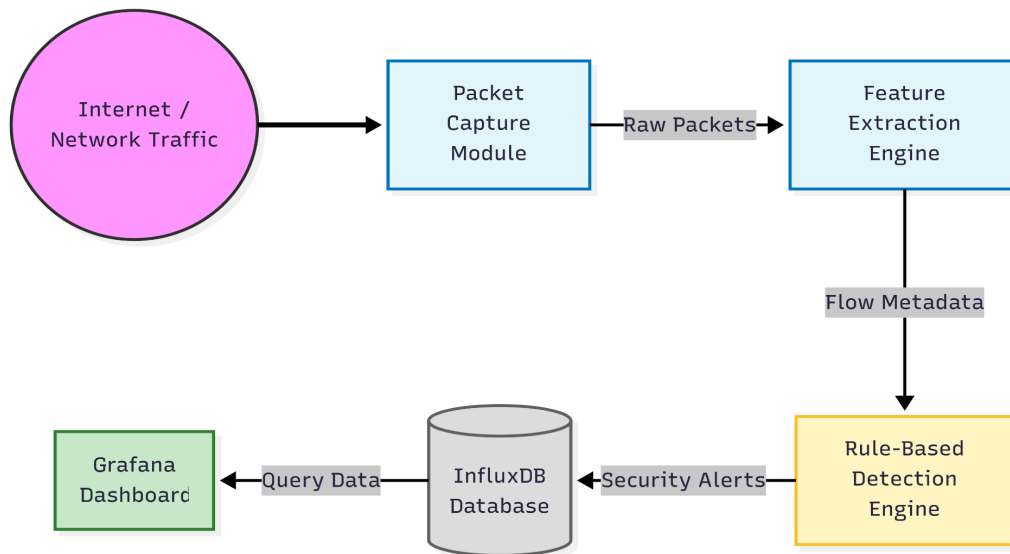
The Lightweight IDS is a modular network security solution designed to run efficiently on consumer-grade hardware. The system operates through a three-stage data processing pipeline:

1. **Traffic Capture:** Raw network packets are captured in real-time. The system reads packet headers to identify source/destination IPs and ports while ignoring payloads to maintain high performance.
2. **Feature Extraction & Flow Analysis:** Captured packets are aggregated into "flows". The system calculates statistical metrics such as "packets per second", "SYN flag count", and "flow duration". These metrics are essential for detecting volumetric attacks like DoS or scanning activities without inspecting packet contents.
3. **Rule-Based Detection Engine:** The extracted statistics are compared against a set of configurable rules and thresholds (e.g., *"If a single IP sends >100 SYN packets in 1 second, flag as SYN Flood"*). If a threshold is breached, an alert is generated immediately.
4. **Reporting:** Alerts and traffic statistics are logged into a time-series database (InfluxDB) and visualized on a Grafana dashboard. The architecture is designed to support a future 'Tier 2' Machine Learning module for anomaly detection, which will run in parallel with the Rule Engine.

3. System Architecture

3.1 Architectural Design

The architecture follows a strictly linear "Pipe and Filter" pattern. Data flows from the capture module directly to the rule engine.



(Future Tier 2 Pipeline is excluded from this diagram since its TBD)

3.2 Decomposition Description

The system is logically divided into four distinct subsystems to ensure modularity and operational efficiency.

3.2.1 Packet Capture & Preprocessing

This subsystem acts as the system's interface with the physical network. It utilizes the Scapy library to intercept raw network frames in promiscuous mode. Its primary responsibility is to filter out irrelevant local traffic (via BPF filters) and parse raw binary data into manipulatable packet objects, stripping payloads to optimize performance before forwarding headers to the processing engine.

3.2.2 Flow Feature Extraction Engine

Since the system performs Encrypted Traffic Analysis (ETA), this module transforms individual packets into aggregated "flows" identified by a 5-tuple hash (IPs, Ports, Protocol). It maintains an in-memory state of active connections and calculates real-time statistics—such as flow duration, packet rates, and SYN/ACK flag counts—which serve as the mathematical basis for detection logic.

3.2.3 Rule-Based Detection Engine

This is the deterministic core of the system. It loads security rules from an external configuration file (Rules.Yaml) and evaluates the extracted flow metrics against predefined thresholds. The engine is designed to instantly flag specific patterns, such as volumetric anomalies (DoS floods) or connection spikes (Port Scans), generating structured Alert objects when a threshold is breached.

3.2.4 Storage & Visualization

This subsystem handles data persistence and user interaction. It writes high-velocity flow metrics and alert logs to InfluxDB, a time-series database optimized for write-heavy workloads. The user interface is provided by Grafana, which queries this database to present real-time dashboards of network health and intrusion alerts.

3.3 Design Rationale

The architectural decisions for the Lightweight Rule-Based IDS were driven by the constraints of small-scale networks and the specific requirements of the project proposal.

3.3.1 Hybrid Detection Strategy (Phased Approach)

The system utilizes a Phased Hybrid approach as defined in the SRS:

- **Phase 1 (Current): Rule-Based Detection.** Selected for Release 1.0 because it offers deterministic, explainable alerts with $O(1)$ complexity, ensuring lightweight operation on constrained hardware. It addresses known threats (DoS, Scans) with zero false positives.
- **Phase 2 (Future): Machine Learning.** The architecture allows for the future integration of an Unsupervised ML model (Isolation Forest). To remain lightweight, this future module is designed to utilize **Recursive Clustering** and **Monte Carlo Cross-Entropy** for data condensation (Almalawi method), preventing resource exhaustion.

3.3.2 Why Flow-Based Analysis (ETA)?

Modern network traffic is increasingly encrypted (HTTPS/TLS). Traditional Deep Packet Inspection (DPI) is blind to encrypted payloads and computationally expensive.

- **Privacy:** Flow analysis respects user privacy by not inspecting the content of communications.
- **Visibility:** Attacks like DoS floods and Port Scans are clearly visible in traffic metadata (headers, sizes, timing) regardless of encryption.

3.3.3 Why Python & Scapy?

- **Rapid Development:** Python offers extensive libraries for network programming.
- **Scapy:** Although slower than C-based sniffers, Scapy provides unmatched flexibility for parsing and crafting packets, making it ideal for the prototyping phase of this project.

3.3.4 Why InfluxDB & Grafana?

Network data is inherently time-series data. Relational databases (like MySQL) struggle with the write loads of real-time network monitoring. InfluxDB is optimized for this exact use case. Grafana provides a production-ready frontend without requiring custom web development, satisfying the "Dashboard" requirement efficiently.

4. Data Design

4.1 Data Description

The system handles data in two states: **Transient (In-Memory)** and **Persistent (Storage)**.

- **Transient Data:** Raw packets captured by the sniffer are processed immediately in RAM to extract metadata. To ensure the system remains lightweight and respects privacy, raw packet payloads are discarded immediately after header parsing and are never stored on the disk.
- **Persistent Data:** Only critical security information—specifically, the generated alerts and aggregated flow metrics—is written to the database. This approach minimizes storage requirements and I/O overhead.

4.2 Data Dictionary

The following table defines the data fields (features) extracted from network flows. These fields serve as the input for the Rule-Based Detection Engine.

Field Name	Data Type	Unit	Description
timestamp	DateTime	N/A	The exact date and time (UTC) when the flow was recorded.
src_ip	String	N/A	The IPv4 address of the sender.
dst_ip	String	N/A	The IPv4 address of the receiver.
src_port	Integer	N/A	The transport layer source port (e.g., 443).
dst_port	Integer	N/A	The transport layer destination port.
protocol	String	N/A	Protocol used (TCP, UDP, ICMP).
flow_duration	Float	Seconds	The time elapsed between the first and last packet of the flow.
packet_count	Integer	Count	Total number of

			packets transferred in the flow.
byte_count	Integer	Bytes	Total size of the payload transferred.
syn_flag_count	Integer	Count	Number of packets with the SYN flag set (Used for SYN Flood detection).
ack_flag_count	Integer	Count	Number of packets with the ACK flag set.
alert_type	String	N/A	The name of the rule triggered (e.g., "Potential Port Scan").
severity	String	N/A	Risk level of the alert (Low, Medium, High, Critical).

4.3 Database Schema Strategy (InfluxDB)

Since InfluxDB is a NoSQL time-series database, it does not use traditional tables. Instead, data is organized into Measurements, Tags, and Fields:

- **Measurement:** network_traffic
- **Tags (Indexed):** src_ip, dst_ip, alert_type, severity (Allows fast filtering in Grafana).
- **Fields (Non-Indexed Values):** packet_count, byte_count, flow_duration (Used for calculations and graphing).

4.4 Mathematical Model of Feature Extraction

The system transforms raw packet streams into statistical flow features. To detect anomalies such as DoS floods or scanning activities without payload inspection, we utilize specific mathematical aggregations. The following formulas define how the core metrics are calculated in the FlowProcessor engine.

4.4.1 Inter-Arrival Time (IAT)

The Inter-Arrival Time measures the time gap between two consecutive packets in a flow. It is crucial for distinguishing between human traffic (irregular IAT) and automated bot traffic (regular or extremely short IAT).

Let t_i be the arrival timestamp of the i -th packet in a flow. The IAT for the k -th interval is defined as:

$$IAT_k = t_k - t_{k-1}$$

The **Mean IAT (μIAT)** for a flow with n packets is calculated to detect high-frequency injection attacks:

$$\mu_{IAT} = \frac{1}{n-1} \sum_{k=2}^n (t_k - t_{k-1})$$

- *Usage in Detection:* If $\mu IAT \approx 0$, it indicates a flooding attack (e.g., UDP Flood).

4.4.2 Flow Throughput (Rates)

Throughput metrics define the volumetric density of the connection. Let B be the total bytes transferred and D be the flow duration ($t_{last} - t_{first}$).

The **Byte Rate (RB)** is defined as:

$$R_B = \frac{\sum_{i=1}^n size(packet_i)}{t_n - t_1}$$

The **Packet Rate (RP)** is defined as:

$$R_P = \frac{n}{t_n - t_1}$$

- *Usage in Detection:* A sudden spike in RP combined with small packet sizes typically indicates a DoS attempt.

4.4.3 TCP Flag Ratios

To detect TCP-based anomalies like SYN Floods, we calculate the ratio of SYN packets to the total packets in the flow.

Let $NSYN$ be the count of packets where the SYN flag is set to 1.

$Ratio_{SYN} = n_{NSYN}$

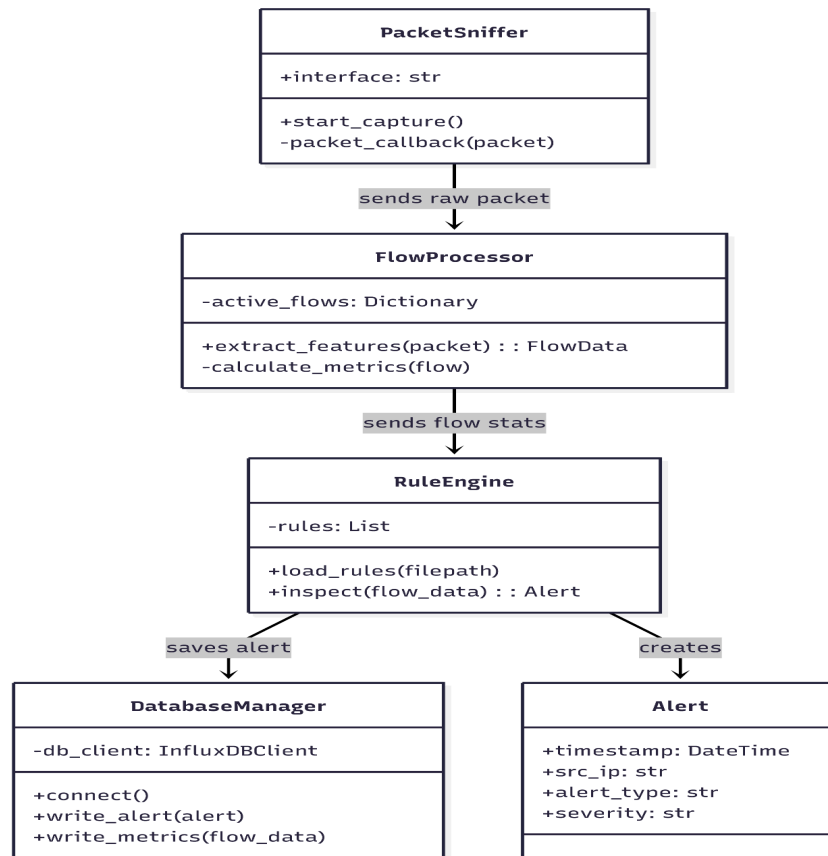
- *Usage in Detection:* In a normal handshake, $Ratio_{SYN}$ is low (usually 1 or 2 per flow). If $Ratio_{SYN} > 0.8$ (i.e., 80% of packets are SYN), it is flagged as a SYN Flood.

5. Component Design

The system is designed using a modular Object-Oriented Programming (OOP) approach. Each functional subsystem described in the architecture is implemented as a distinct Python class. This ensures encapsulation and simplifies unit testing.

5.1 System Class Diagram

The following Unified Modeling Language (UML) Class Diagram illustrates the static structure of the system, showing the attributes and operations of each class and the relationships between them.



(Future Tier 2 Pipeline is excluded from this diagram since its TBD)

5.2 Component Descriptions

5.2.1 Class: PacketSniffer

- **Responsibility:** Acts as the entry point for network traffic. It wraps the Scapy sniffing functions to capture packets efficiently.
- **Attributes:**
 - interface (String): The network interface name to bind (e.g., "eth0").
- **Methods:**
 - start_capture(): Initiates the sniffing loop in promiscuous mode.
 - packet_callback(packet): A callback function triggered for every packet. It performs basic filtering (dropping non-IP packets) before passing data to the processor.

5.2.2 Class: FlowProcessor

- **Responsibility:** Manages the state of active network connections and calculates statistical features required for detecting anomalies in encrypted traffic.
- **Attributes:**

- active_flows (Dictionary): A hash map where keys are 5-tuples and values are Flow objects.
- **Methods:**
 - extract_features(packet): Identifying the flow a packet belongs to and updating its counters (byte count, packet count).
 - calculate_metrics(): Computes derived rates (e.g., Packets per Second) to prepare the feature vector for the detection engine.

5.2.3 Class: RuleEngine

- **Responsibility:** The deterministic core that validates flow metrics against security policies.
- **Attributes:**
 - rules (List): A collection of threshold definitions loaded from the configuration file.
- **Methods:**
 - load_rules(filepath): Parses the YAML configuration file to initialize detection logic.
 - inspect(flow_data): Comparison logic.
 - *Logic:* It iterates through all active rules. If flow_data.syn_count > rule.syn_threshold, it triggers an alert.

5.2.4 Class: DatabaseManager

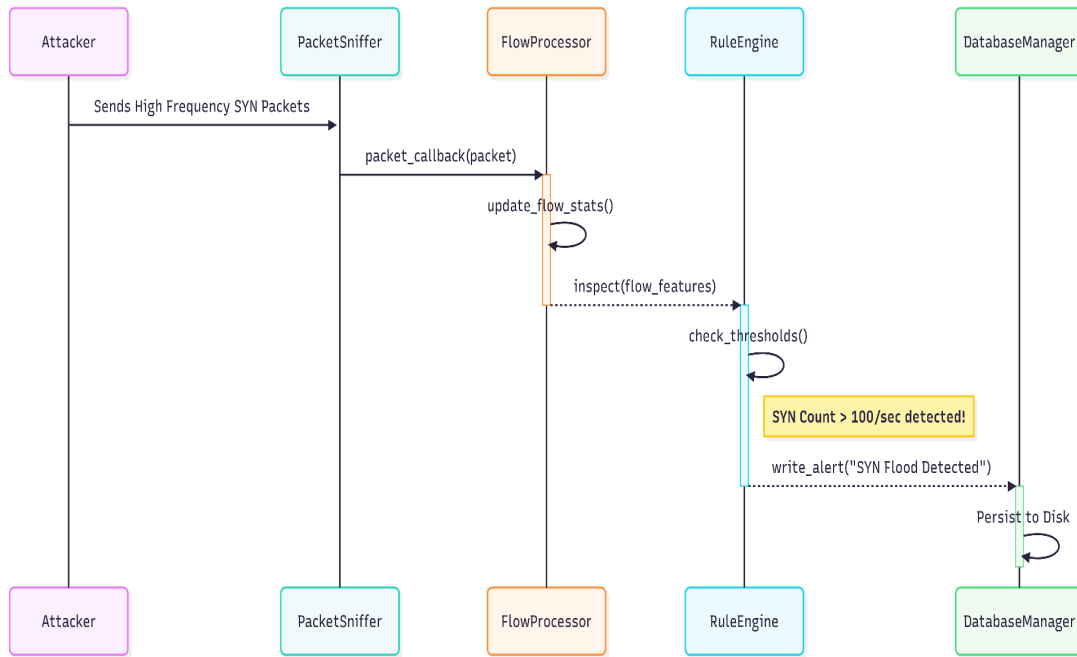
- **Responsibility:** Handles all I/O operations with persistent storage.
- **Attributes:**
 - db_client: The active connection object to the InfluxDB instance.
- **Methods:**
 - write_alert(alert): Formats an Alert object into InfluxDB Line Protocol and writes it to the alerts measurement.
 - write_metrics(flow_data): Writes raw traffic statistics to the traffic_stats measurement for dashboard visualization.

5.2.5 Class: MLEngine (Future/Conditional)

- **Responsibility:** Implements the Tier 2 anomaly detection logic.
- **Attributes:**
 - model: The serialized Scikit-learn model.
- **Methods:**
 - predict_anomaly(flow_data): Transforms the flow features into a vector and queries the model. If the score < -0.6 (anomaly), it generates an Alert of type ML_ANOMALY.

5.3 Dynamic Behavior (Sequence Diagram)

The following Sequence Diagram details the runtime interaction between objects when a "SYN Flood" attack is detected.



5.4 Algorithmic Specifications

This section provides the pseudo-code for the core processing logic, illustrating how the system handles continuous network traffic and executes detection rules in real-time.

5.4.1 Algorithm 1 Main Packet Processing Loop

This algorithm runs inside the PacketSniffer and FlowProcessor modules. It ensures that every packet is mapped to a correct flow session.

Input: Stream of raw network packets (P)

Output: Aggregated Flow Features (F)

```

1: Initialize FlowTable (Hash Map)
2: FOR EACH packet P in Stream DO:
3:     IF P is not IP/TCP/UDP/ICMP THEN
4:         Continue (Ignore packet)
5:     END IF
6:
7:     # Generate 5-tuple Key
8:     Key = Hash(P.srcIP, P.dstIP, P.srcPort, P.dstPort, P.proto)
9:
10:    IF Key exists in FlowTable THEN
11:        Flow = FlowTable[Key]
12:        Flow.packet_count += 1
13:        Flow.byte_count += P.size
14:        Flow.duration = CurrentTime - Flow.start_time
  
```

```
15:     Update_TCP_Flags(Flow, P.flags)
16: ELSE
17:     Create New Flow object
18:     Flow.start_time = CurrentTime
19:     FlowTable.insert(Key, Flow)
20: END IF
21:
22: # Trigger Detection Logic
23: CALL RuleEngine.Inspect(Flow)
24: END FOR
```

5.4.2 Algorithm 2 Rule-Based Inspection Logic

This algorithm describes the deterministic decision-making process within the RuleEngine.

Input: Flow Feature Vector (F), List of Rules (R)

Output: Alert (A) OR Null

```
1: FOR EACH Rule r in R DO:
2:     is_violation = False
3:
4:     # Check Thresholds based on Rule Type
5:     SWITCH r.type:
6:         CASE "VOLUMETRIC":
7:             IF F.packet_rate > r.pps_threshold THEN
8:                 is_violation = True
9:             END IF
10:
11:         CASE "PROTOCOL_ANOMALY":
12:             IF F.syn_ratio > r.syn_threshold THEN
13:                 is_violation = True
14:             END IF
15:
16:         CASE "SCANNING":
17:             # Requires history check of Source IP
18:             IF Count_Unique_Ports(F.srcIP) > r.port_limit THEN
19:                 is_violation = True
20:             END IF
21:     END SWITCH
22:
23:     IF is_violation IS True THEN
24:         Alert A = CreateAlert(Timestamp, F.srcIP, r.name, r.severity)
25:         CALL Database.Write(A)
26:         RETURN A # Stop checking other rules for efficiency
27:     END IF
28: END FOR
29: RETURN Null
```

6. Human Interface Design

6.1 Overview of User Interface

The user interface is a web-based dashboard powered by Grafana, which connects to the InfluxDB backend. It is designed to provide network administrators with a "single pane of glass" view of the network's security posture. The dashboard updates in real-time (every 5 seconds) to reflect the latest traffic statistics and security alerts captured by the Rule-Based Engine.

6.2 Screen Images

TBD

6.3 Screen Objects and Actions

The dashboard is divided into three logical panels, allowing the user to monitor volumetric traffic and specific security incidents simultaneously.

6.3.1 Panel A: Network Throughput & Latency

- **Object Type:** Time-Series Graph (Line Chart).
- **Data Source:** traffic_stats measurement in InfluxDB.
- **X-Axis:** Time (Last 15 minutes window).
- **Y-Axis:** Bits per Second (bps) and Packets per Second (pps).
- **Action:** Hovering over the graph displays the exact throughput values for a specific timestamp. This helps administrators visually identify DoS attacks (sudden spikes in the graph).

6.3.2 Panel B: Recent Alerts Log

- **Object Type:** Table View.
- **Data Source:** alerts measurement.
- **Columns:**
 - Time: Timestamp of the detection.
 - Source IP: The attacker's IP address.
 - Rule Name: The specific rule that was violated (e.g., "SYN_Flood_Threshold_Exceeded").
 - Severity: Color-coded severity level (Yellow for Warning, Red for Critical).
- **Action:** Rows are sorted descending by time. This allows the admin to see the most recent threats immediately.

6.3.3 Panel C: Protocol Distribution

- **Object Type:** Pie Chart.
- **Data Source:** Aggregated flow protocols.
- **Description:** Shows the percentage breakdown of TCP vs. UDP vs. ICMP traffic.

- **Purpose:** Helps identify protocol anomalies (e.g., an unusual surge in ICMP traffic indicating a Ping Flood).

7. Requirements Matrix

Req ID	Requirement Description	Implementing Component	Priority
R1	The system shall capture real-time network traffic from a local interface.	PacketSniffer Class	High
R2	The system shall process packets to extract flow-based statistical features (metadata) without payload inspection.	FlowProcessor Class	High
R3	The system must allow administrators to define security rules and thresholds via a configuration file.	RuleEngine Class (rules.yaml)	High
R4	The system shall detect "Port Scanning" behavior based on connection frequency thresholds.	RuleEngine (Threshold Logic)	High
R5	The system shall detect "SYN Flood" attacks based on flag ratios in the flow.	RuleEngine (Signature Logic)	High
R6	The system shall persist alerts to a database and visualize them on a dashboard.	DatabaseManager & Grafana	Medium
R7	The system architecture shall support a future ML module for anomaly detection.	MLEngine	Low (Future)
R8	The ML training pipeline must use data condensation to minimize RAM usage.	Offline Training Script	Low (Future)

8. Appendices

Appendix A: Sample Configuration File (rules.yaml) The following is an example of the configuration file used by the RuleEngine to load detection logic.

```
# Rule Definitions for Lightweight IDS

global_settings:
  interface: "eth0"
  log_level: "INFO"

rules:
  - rule_id: 101
    name: "Potential SYN Flood"
    description: "Detects high rate of SYN packets without ACKs"
    thresholds:
      syn_count_per_sec: 100
      syn_ack_ratio: 0.9
    severity: "CRITICAL"

  - rule_id: 102
    name: "Port Scanning Activity"
    description: "Detects single source connecting to multiple ports"
    thresholds:
      unique_ports_per_sec: 20
    severity: "HIGH"
```

9. Test Strategy and Validation

9.1 Test Environment Setup

To validate the functionality and performance of the Lightweight IDS, a controlled testbed environment will be established. The environment consists of three isolated virtual machines (VMs) connected via a virtual internal network to prevent leakage of malicious traffic to the public internet.

- **Victim Machine:** A Linux server (Ubuntu 22.04) running standard services (HTTP via Apache, SSH). This machine serves as the target for attacks.
- **Attacker Machine:** A Kali Linux instance equipped with penetration testing tools (hping3, nmap, Hydra) to generate malicious traffic.
- **IDS Machine:** The device hosting the developed Python-based IDS, configured to listen on the interface in promiscuous mode (mirror port).

9.2 Attack Simulation Tools & Scenarios

The following industry-standard tools will be used to generate synthetic attacks and verify if the RuleEngine triggers the expected alerts.

9.2.1 Scenario A: Port Scanning Detection

- Objective: Verify if the system detects horizontal and vertical scanning.
- Tool: Nmap (Network Mapper).

Command:

```
# Stealth SYN Scan (Half-open)
nmap -sS -p 1-1000 <Victim_IP>
# UDP Scan (slower, but essential for validation)
nmap -sU -p 53,161 <Victim_IP>
```

Expected Outcome: The IDS should generate an alert with RuleID: 102 (Port Scanning Activity) when the number of unique destination ports exceeds the defined threshold (e.g., >20 ports/sec).

9.2.2 Scenario B: DoS / SYN Flood Detection

- Objective: Validate the detection of volumetric denial-of-service attacks targeting the TCP handshake mechanism.
- Tool: hping3 (Packet Generator).

Command:

```
# Flood target with SYN packets (simulating 10,000 pps)
sudo hping3 -S --flood -V -p 80 <Victim_IP>
```

Expected Outcome: The FlowProcessor should observe a high Ratio_SYN (> 0.9). The RuleEngine must trigger a Critical Alert (RuleID: 101) within 2 seconds of attack initiation.

9.2.3 Scenario C: Network Stress Testing (Benign Traffic)

- Objective: Ensure the IDS does not crash or produce False Positives under heavy *normal* traffic load.
- Tool: iPerf3.

Command:

```
# Server side (Victim)
iperf3 -s
# Client side (Attacker acting as normal user)
iperf3 -c <Victim_IP> -t 60 -b 100M
```

Expected Outcome: The system should process the high throughput (100 Mbps) without triggering any alerts, verifying the False Positive Rate (FPR) is near zero for legitimate heavy loads.

9.3 Performance Evaluation Metrics

To objectively measure the "Lightweight" aspect of the system, the following resource consumption metrics will be recorded during the stress tests:

- CPU Usage: Average percentage of CPU cores utilized by the PacketSniffer process.
- Memory Footprint: RAM consumption (MB) of the internal FlowTable hash map.
- Detection Latency: The time difference (Δt) between the arrival of the first malicious packet (t_{start}) and the timestamp of the generated alert (t_{alert}).

$$\Delta t = t_{alert} - t_{start}$$

9.4 Validation Metrics

The accuracy of the Rule-Based Engine will be quantified using a Confusion Matrix approach based on the CIC-IDS-2017 dataset labels:

- True Positive (TP): Attack occurred AND Alert triggered.
- False Positive (FP): No attack AND Alert triggered (False Alarm).
- False Negative (FN): Attack occurred AND No Alert triggered (Missed Detection).
- True Negative (TN): No attack AND No Alert triggered.

The Detection Rate (Recall) will be calculated as:

$$Recall = \frac{TP}{TP + FN}$$